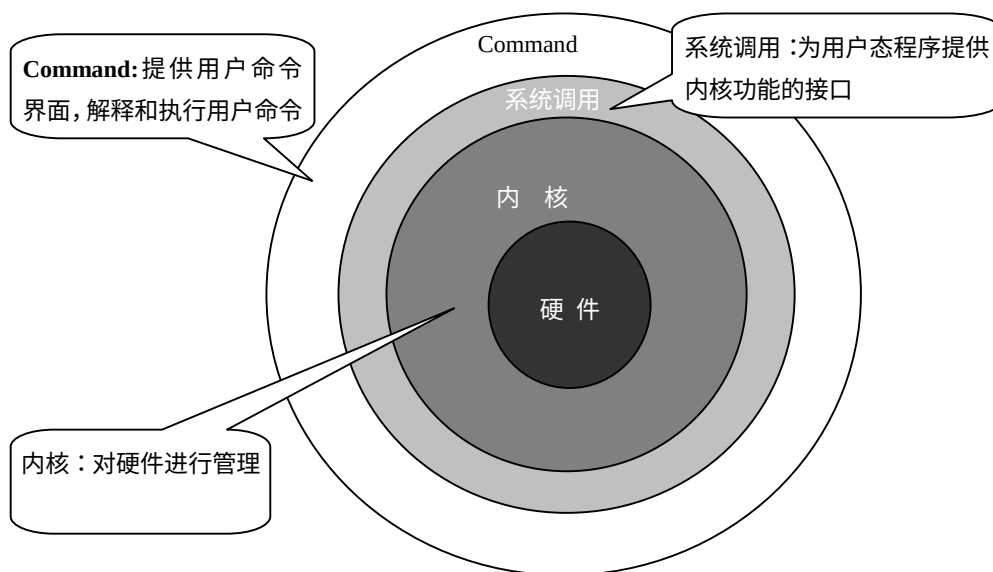


命令解释程序与内核的关系

命令解释程序是用户和系统内核之间的接口程序。对于 Windows 系统来说，由于已经提供了具有良好交互性的图形用户界面，传统的控制台命令解释程序已经很少被广大用户所了解和使用。但是，对于某些应用，例如删除所有扩展名为 tmp 的文件，或是删除某些具有特殊名字的病毒文件，在图形用户界面下很难甚至不能完成，需要通过 Windows 提供的 Command 命令接口来完成。Command 程序是一个命令语言解释器，它拥有自己内建的命令集，用户或其它应用程序都可通过对 Command 程序的调用完成与系统内核的交互。我们可以把系统内核想象成一个球体的中心，Command 命令解释程序就是包围内核的外壳，如图 1-2 所示。



系统调用

应用程序是以函数调用的方式来通知操作系统执行相应的内核功能。操作系统所能够完成的每一个特殊功能通常都有一个函数与其对应，即操作系统把它所能够完成的功能以函数的形式提供给应用程序使用，应用程序对这些函数的调用叫做系统调用，这些函数的集合就是 Windows 操作系统提供给应用程序编程的接口（Application Programming Interface），简称 Windows API 或 Win32 API（注：某些 Win32 API，如管理 Windows 线程的 API 等，它们并没有操纵内核对象，因此不是系统调用。本实验只讨论 API 的使用，不再做进一步区分）。所有在 Win32 平台上运行的应用程序都可以调用这些函数。

使用 Windows API，应用程序可以充分挖掘 Windows 的 32 位操作系统的潜力。Microsoft 的所有 32 位平台都支持统一的 API，包括函数、结构、消息、宏及接口。使用 Windows API 不但可以开发出在各种平台上都能运行的应用程序，而且也可以充分利用每个平台特有的功能和属性。

Windows 的相关 API 都可以在 MSDN 中查到，包括定义、使用方法等等。下面简单介绍一下本次实验中涉及到的 Windows API。

1. GetCurrentDirectory 函数

功能说明

查找当前进程的当前目录，调用成功返回装载到 lpBuffer 的字节数。

格式

```
DWORD GetCurrentDirectory(  
    DWORD nBufferLength,  
    LPTSTR lpBuffer  
)
```

参数说明

nBufferLength：缓冲区的长度。

lpBuffer：指定一个预定义字符串，用于装载当前目录。

2. WaitForSingleObject 函数

功能说明

等待一个事件信号直至该信号出现或是超时。若有信号则返回 WAIT_OBJECT_0，若等待超过 dwMilliseconds 时间还是无信号则返回 WAIT_TIMEOUT。

格式

```
DWORD WaitForSingleObject(  
    HANDLE hHandle,  
    DWORD dwMilliseconds  
)
```

参数说明

hHandle：事件的句柄。

dwMilliseconds：最大等待时间，以 ms 计时。

3. SetCurrentDirectory

功能说明

设置当前目录，非零表示成功，零表示失败。

格式

```
BOOL SetCurrentDirectory(  
    LPCTSTR lpPathName  
)
```

参数说明

lpPathName：新当前目录的路径。

4. FindFirstFile 函数

功能说明

该函数到一个文件夹（包括子文件夹）去搜索指定文件，由这个函数返回的句柄可以作为一个参数用于 FindNextFile 函数。这样一来，就可以方便的枚举出与 lpFileName 参数指定的文件名相符的所有文件。

格式

```
HANDLE FindFirstFile(  
    LPCTSTR lpFileName,  
    LPWIN32_FIND_DATA lpFindFileData
```

)

参数说明

lpFileName：文件名字符串。

lpFindFileData：指向一个用于保存文件信息的结构体。

5. FindNextFile 函数

功能说明

继续查找 FindFirstFile 函数搜索后的文件。由这个函数返回的句柄可以作为一个参数用于 FindNextFile 函数。这样一来，就可以方便的枚举出与 lpFileName 参数指定的文件名相符的所有文件。

格式

```
BOOL FindNextFile(  
    HANDLE hFindFile,  
    LPWIN32_FIND_DATA lpFindFileData  
)
```

参数说明

hFindFile：前一个搜索到的文件的句柄。

lpFindFileData：指向一个用于保存文件信息的结构体。

6. GetVolumeInformation 函数

功能说明

用于获取磁盘相关信息。

格式

```
BOOL GetVolumeInformation(  
    LPCTSTR lpRootPathName,  
    LPTSTR lpVolumeNameBuffer,  
    DWORD nVolumeNameSize,  
    LPDWORD lpVolumeSerialNumbe,  
    LPDWORD lpMaximumComponentLength,  
    LPDWORD lpFileSystemFlags,  
    LPTSTR lpFileSystemNameBuffer,  
    DWORD nFileSystemNameSize  
)
```

参数说明

lpRootPathName：磁盘驱动器代码字符串。

lpVolumeNameBuffer：磁盘驱动器卷标名称。

nVolumeNameSize：磁盘驱动器卷标名称长度。

lpVolumeSerialNumbe：磁盘驱动器卷标序列号。

lpMaximumComponentLength：系统允许的最大文件长度。

lpFileSystemFlags：文件系统标识。

lpFileSystemNameBuffer：文件系统名称。

nFileSystemNameSize：文件系统名称长度。

7. GetDiskFreeSpaceEx 函数

功能说明

获取与一个磁盘的组织以及剩余空间容量有关的信息。

格式

```
BOOL GetDiskFreeSpaceEx(  
    LPCTSTR lpRootPathName,  
    PULARGE_INTEGER lpFreeBytesAvailableToCaller,  
    PULARGE_INTEGER lpTotalNumberOfBytes,  
    PULARGE_INTEGER lpTotalNumberOfFreeBytes,  
)
```

参数说明

lpRootPathName：不包括卷名的磁盘根路径名。
lpFreeBytesAvailableToCaller：调用者可用的字节数量。
lpTotalNumberOfBytes：磁盘上的总字节数。
lpTotalNumberOfFreeBytes：磁盘上可用的字节数。

8. FileTimeToLocalFileTime 函数

功能说明

将一个 FILETIME 结构转换成本地时间。

格式

```
BOOL FileTimeToLocalFileTime(  
    const FILETIME* lpFileTime,  
    LPFILETIME lpLocalFileTime  
)
```

参数说明

lpFileTime：包含了 UTC 时间信息的一个结构。
lpLocalFileTime：用于装载转换过后的本地时间的结构体。

9. FileTimeToSystemTime 函数

功能说明

根据一个 FILETIME 结构的内容，装载一个 SYSTEMTIME 结构。

格式

```
BOOL FileTimeToSystemTime(  
    const FILETIME* lpFileTime,  
    LPSYSTEMTIME lpSystemTime  
)
```

参数说明

lpFileTime：包含了文件时间的一个结构。
lpSystemTime：用于装载系统时间信息的一个结构体。

10. CreateToolhelp32Snapshot 函数

功能说明

为指定的进程、进程使用的堆（HEAP）、模块（MODULE）、线程（THREAD）建立一个快照（snapshot）。快照建立成功则返回快照的句柄，失败则返回 INVALID_HANDLE_VALUE。

格式

```
HANDLE WINAPI CreateToolhelp32Snapshot(  
    DWORD dwFlags,  
    DWORD th32ProcessID  
)
```

参数说明

dwFlags：指定快照中包含的系统内容。

th32ProcessID：指定将要快照的进程 ID。

11. Process32First 函数

功能说明

Process32First 是一个进程获取函数，当使用 CreateToolhelp32Snapshot 函数获得当前运行进程的快照后，可以利用 Process32First 函数来获得第一个进程的句柄。

格式

```
BOOL WINAPI Process32First(  
    HANDLE hSnapshot,  
    LPPROCESSENTRY32 lppe  
)
```

参数说明

hSnapshot：快照句柄。

lppe：指向一个保存进程快照信息的 PROCESSENTRY32 结构体。

12. Process32Next 函数

功能说明

获取进程快照中下一个进程信息。

格式

```
BOOL WINAPI Process32Next(  
    HANDLE hSnapshot,  
    LPPROCESSENTRY32 lppe  
)
```

参数说明

hSnapshot：由 Process32First 函数或 Process32Next 函数获得的快照句柄。

lppe：指向一个保存进程快照信息的 PROCESSENTRY32 结构体。

13. OpenProcess 函数

功能说明

OpenProcess 函数打开一个已存在的进程对象。如成功，返回值为指定进程的打开句柄。如失败，返回值为空。

格式

```
HANDLE OpenProcess(  
    DWORD dwDesiredAccess,  
    BOOL bInheritHandle,  
    DWORD dwProcessId  
)
```

参数说明

dwDesiredAccess：权限标识。
bInheritHandle：句柄继承标识。
dwProcessId：进程 id。

14. SetConsoleCtrlHandler 函数

功能说明

添加或删除一个事件钩子（handler）。

格式

```
BOOL SetConsoleCtrlHandler(  
    PHANDLER_ROUTINE HandlerRoutine,  
    BOOL Add  
)
```

参数说明

HandlerRoutine：回调函数的指针。
Add：表示添加或是删除。

15. CreateProcess 函数

功能说明

创建一个新的进程和它的主线程，这个新进程运行指定的可执行文件。

格式

```
BOOL CreateProcess(  
    LPCTSTR lpApplicationName,  
    LPTSTR lpCommandLine,  
    LPSECURITY_ATTRIBUTES lpProcessAttributes,  
    LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    BOOL bInheritHandles,  
    DWORD dwCreationFlags,  
    LPVOID lpEnvironment,  
    LPCTSTR lpCurrentDirectory,  
    LPSTARTUPINFO lpStartupInfo,  
    LPPROCESS_INFORMATION lpProcessInformation  
)
```

参数说明

lpApplicationName：指定可执行模块的字符串。
lpCommandLine：指定要运行的命令行。
lpProcessAttributes：决定返回句柄能否被继承。
lpThreadAttributes：决定返回句柄能否被继承。
bInheritHandles：表示新进程是否从调用进程处继承了句柄。
dwCreationFlags：控制优先类和进程的创建标志。
lpEnvironment：指向一个新进程的环境块。
lpCurrentDirectory：指定子进程的工作路径。
lpStartupInfo：决定新进程的主窗体显示方式的结构体。
lpProcessInformation：接收新进程的识别信息。

16. GetExitCodeProcess 函数

功能说明

获取一个已中断进程的退出代码。

格式

```
BOOL GetExitCodeProcess(  
    HANDLE hProcess,  
    LPDWORD lpExitCode  
)
```

参数说明

hProcess 为进程句柄。

lpExitCode 指向接受退出码的变量。

17. TerminateProcess 函数

功能说明

以给定的退出码终止一个进程。

格式

```
BOOL TerminateProcess(  
    HANDLE hProcess,  
    UINT uExitCode  
)
```

参数说明

hProcess：进程句柄。

uExitCode：进程退出码。